

CANLIFAL — FLUTTER GELİŞTİRME PROMPT'U

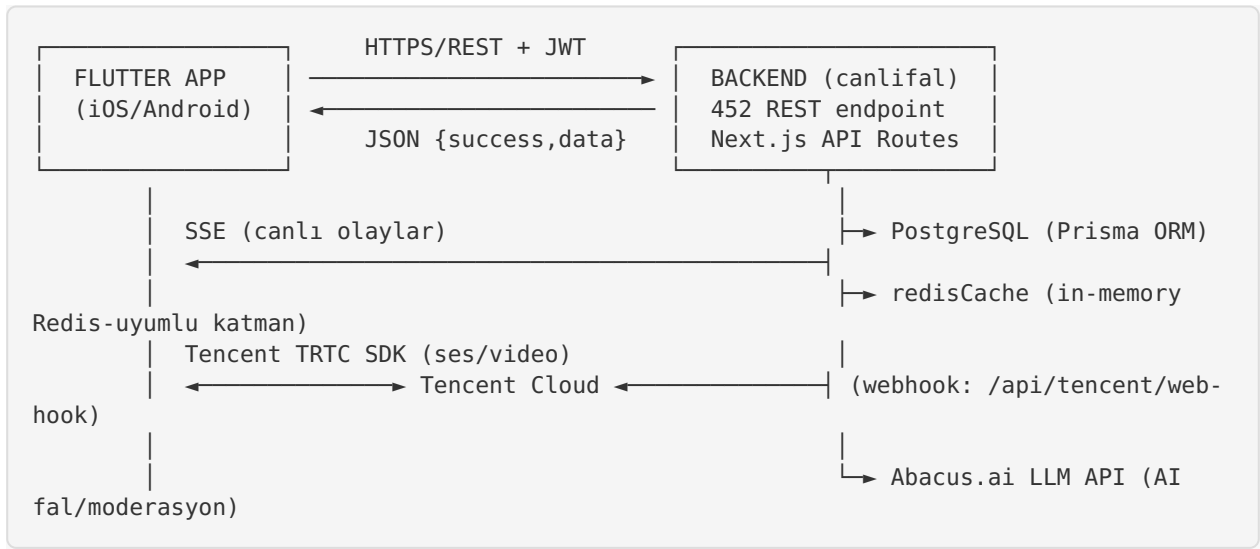
PART 1 — SİSTEM MİMARİSİ

Kaynak: Bu doküman %100 gerçek backend koduna dayalıdır. Tahmini/varsayımsal API yoktur.

Backend: <https://canlifal.com> (production)

Auth Secret: NEXTAUTH_SECRET ile imzalanan JWT (backend ile paylaşılan gizli anahtar)

1. GENEL MİMARİ: Flutter + Backend + Abacus.ai



Üç katman:

1. **Flutter App** — Sunum katmanı. Hiçbir iş mantığı içermez, sadece API çağırır.
2. **Backend (canlifal.com)** — Tüm iş mantığı, auth, ödeme, kredi, fal, canlı yayın yönetimi.
3. **Abacus.ai** — LLM API'leri (AI fal yorumu, AI moderatör, AI öneriler). Backend üzerinden çağrılır, Flutter doğrudan Abacus.ai'ye bağlanmaz.

Kritik kural: Flutter hiçbir gizli anahtarı (TRTC secret, Abacus API key, DB) tutmaz. Tüm hassas işlemler backend'de yapılır. Flutter yalnızca JWT taşır.

2. API STANDARTLARI

2.1 Base URL

```
const String kBaseUrl = 'https://canlifal.com';
const String kApiUrl = '$kBaseUrl/api';
```

2.2 Standart yanıt formatı

Tüm mobil-uyumlu endpoint'ler şu zarfı döner:

Başarılı:

```
{ "success": true, "data": { ... } }
```

Hatalı:

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Oturum açmanız gerekiyor" } }
```

Not: Bazı eski/web-paylaşımli endpoint'ler düz obje döner (ör. `/api/video-streams` → `{ streams, items, pagination }`). Flutter tarafında her iki formatı da tolere eden parse helper kullan:

```
dart
// list parse örneği
final list = data is List ? data : (data['streams'] ?? data['items'] ?? []);
```

2.3 HTTP başlıkları (her istekte)

```
final headers = {
  'Content-Type': 'application/json',
  'Authorization': 'Bearer $accessToken', // korumalı endpoint'ler için
};
```

2.4 HTTP durum kodları

Kod	Anlam
200	Başarılı
400	Geçersiz istek (eksik parametre)
401	Yetkisiz (token yok/geçersiz/süresi dolmuş)
403	Yasak (yetki yetersiz)
404	Bulunamadı
429	Rate limit aşıldı
500	Sunucu hatası

2.5 Hata kodları (error.code)

Backend anlamlı `code` alanı döner: `UNAUTHORIZED`, `MISSING_ROOM_ID`, `TRTC_NOT_CONFIGURED`, `INSUFFICIENT_CREDITS`, `FORBIDDEN` vb. Flutter bu kodlara göre kullanıcıya Türkçe mesaj göster.

3. JWT KİMLİK DOĞRULAMA

3.1 Token yapısı (backend lib/mobile-auth.ts gerçeği)

- **Access Token:** geçerlilik **7 gün** (7d)
- **Refresh Token:** geçerlilik **30 gün** (30d)
- İmza: NEXTAUTH_SECRET (HMAC), jsonwebtoken ile.
- Payload: { userId, email, role, type: 'access' | 'refresh' }

3.2 Auth endpoint'leri (gerçek)

Endpoint	Metod	Amaç
/api/auth/mobile-register	POST	E-posta/şifre ile kayıt
/api/auth/mobile-login	POST	E-posta/şifre ile giriş → access+refresh token
/api/auth/mobile-refresh	POST	Refresh token ile yeni access token
/api/auth/mobile-google	POST	Google SSO ile giriş
/api/auth/mobile-apple	POST	Apple ile giriş
/api/auth/mobile-tiktok	POST	TikTok ile giriş

3.3 Giriş akışı (Flutter)

```
// 1. Login
POST /api/auth/mobile-login
body: { "email": "...", "password": "..." }
👉 { success: true, data: { accessToken, refreshToken, user: {...} } }

// 2. accessToken + refreshToken → flutter_secure_storage'a kaydet
// 3. Her korumalı istekte: Authorization: Bearer <accessToken>
// 4. 401 alınca → /api/auth/mobile-refresh ile yenile → isteği tekrarla
// 5. refresh de 401 verirse → login ekranına yönlendir
```

3.4 Güvenli saklama

```
// flutter_secure_storage kullan (SharedPreferences DEĞİL)
final storage = FlutterSecureStorage();
await storage.write(key: 'accessToken', value: accessToken);
await storage.write(key: 'refreshToken', value: refreshToken);
```

3.5 Otomatik yenileme (interceptor deseni)

Dio interceptor ile 401 yakalandığında refresh yap, orijinal isteği tekrarla. Aynı anda birden fazla 401 gelirse tek refresh yap (mutex/lock ile).

4. SSE (Server-Sent Events) — CANLI OLAYLAR

Backend gerçek zamanlı olayları SSE ile push eder. Flutter bunları dinleyerek anlık UI günceller (WebSocket YOK, SSE kullanılır).

4.1 Gerçek SSE endpoint'leri

Endpoint	Yayınlanan olaylar
/api/room/[sessionId]/stream	message , timer_started , time_extended , session_ended
/api/video-streams/[streamId]/stream	streamMessage , streamGift , streamEnded , viewer olayları
/api/chat/rooms/[roomId]/stream	Sesli oda mesaj/koltuk/DJ olayları
/api/fortune-tellers/sessions/stream	Falcıya gelen istek olayları (session_cancelled vb.)
/api/notifications/stream	Anlık bildirimler

4.2 Flutter SSE tüketimi

```
// http paketi ile streaming request
final request = http.Request('GET', Uri.parse('$kApiUrl/room/$sessionId/stream'));
request.headers['Authorization'] = 'Bearer $accessToken';
request.headers['Accept'] = 'text/event-stream';
final response = await client.send(request);
response.stream
  .transform(utf8.decoder)
  .transform(const LineSplitter())
  .listen((line) {
    if (line.startsWith('data:')) {
      final json = jsonDecode(line.substring(5).trim());
      // event tipine göre işle
    }
  });
```

4.3 Dayanıklılık

- Bağlantı koparsa **exponential backoff** ile yeniden bağlan (1s, 2s, 4s... max 30s).
- Uygulama arka plana alınınca SSE'yi kapat, öne gelince yeniden aç.
- SSE + REST hibrit: kritik veriyi periyodik REST polling ile de doğrula (SSE kaçırılan olay için güvenlik ağı).

5. TENCENT RTC (Ses & Video)

Canlı yayın ve sesli sohbet **Tencent TRTC SDK** ile yapılır. Flutter `tencent_trtc_cloud` paketini kullanır.

5.1 Yapılandırma (gerçek env)

- `TRTC_SDK_APP_ID` = **20040423** (backend'de mevcut)
- `NEXT_PUBLIC_TRTC_SDK_APP_ID` — public sdkAppId
- Secret key backend'de saklanır — **Flutter'a asla verilmez.**
- Token geçerlilik: `TRTC_EXPIRE` = 86400 sn (24 saat)

5.2 UserSig alma (gerçek endpoint)

```
POST /api/trtc/token (JWT korumalı)
body: { "roomId": "<oda-id>", "role": "host" | "audience" }

→ { success: true, data: {
  sdkAppId, // 20040423
  userId, // JWT'den gelen kullanıcı id'si
  userSig, // TRTC imzası (backend tls-sig-api-v2 ile üretir)
  roomId,
  expireTime // 86400
}}
```

Alternatif: `POST /api/trtc/usersig` (aynı amaç).

5.3 Flutter TRTC akışı

```
// 1. Backend'den userSig al
final res = await api.post('/trtc/token', {'roomId': roomId, 'role': 'host'});
// 2. TRTC SDK'yı başlat ve odaya gir
await trtcCloud.enterRoom(TRTCParams(
  sdkAppId: res.sdkAppId,
  userId: res.userId,
  userSig: res.userSig,
  roomId: int.parse(roomId), // veya strRoomId
  role: TRTCCLoudDef.TRTCRoleAnchor, // host → Anchor, audience → Audience
), TRTCCLoudDef.TRTC_APP_SCENE_LIVE);
```

5.4 Webhook (backend tarafı — Flutter'ı ilgilendiren sonuç)

Backend `POST /api/tencent/webhook` ile Tencent'ten olay alır (imza: HMAC-SHA256, `TRTC_WEBHOOK_KEY`):

- **101** Oda oluşturuldu → stream canlı işaretlenir
- **102** Oda dağıtıldı → stream biter, kullanıcılar offline
- **103** Üye girdi → kullanıcı online / izleyici eklenir (redisCache)
- **104** Üye çıktı → kullanıcı offline / izleyici düşülür
- **105** Rol değişti → loglanır
- **201-206** Ses/video başlat-durdur → loglanır

Flutter bu webhook'u **çağırmaz** — Tencent Cloud çağırır. Flutter yalnızca sonucu SSE/REST ile görür (izleyici sayısı, online durum vb.).

6. PERFORMANS MİMARİSİ

6.1 Cache katmanı (gerçek — redisCache)

Backend'de Redis-uyumlu in-memory cache katmanı var (`lib/cache.ts`). Flutter için REST erişimi:

```
GET /api/cache?op=hgetall&key=... (JWT korumalı)
POST /api/cache body:{ op, key, ... }
```

Desteklenen ops: String, Hash, List, Set, Sorted Set, Pub/Sub, TTL. Online kullanıcı takibi, presence bu katmanda tutulur.

6.2 Flutter performans kuralları

- **Görsel cache:** `cached_network_image` kullan. Avatar/thumbnail'ları diskte ön belleğe al.
- **Liste:** `ListView.builder` + pagination (backend `page / limit` destekler, max limit 100).
- **SSE > polling:** Anlık veri için SSE, geri kalan için 10-30 sn TTL'li REST.
- **Debounce:** Arama/filtre isteklerini 300ms debounce et.
- **Bağlantı havuzu:** Tek Dio instance, keep-alive açık.
- **Optimistic UI:** Beğeni/hediye gibi işlemlerde önce UI güncelle, sonra API'ye gönder, hata olursa geri al.

6.3 Backend performans gerçekleri

- `/api/video-streams` GET → 10 sn cache (yoğun polling koruması).
- Platform ayarları (`getCachedPlatformSetting`) cache'li.
- Ödeme yöntemleri / kredi paketleri cache'li.
- DB indeksleri kritik tablolarda mevcut (`TrtcWebhookLog`, `LiveSession` vb.).

7. FLUTTER PROJE YAPISI ÖNERİSİ (Part 1 için)

```
lib/
├── core/
│   ├── api/
│   │   ├── api_client.dart      # Dio + interceptor (JWT refresh)
│   │   ├── endpoints.dart      # tüm endpoint sabitleri
│   │   └── api_response.dart    # {success,data,error} parse
│   ├── auth/
│   │   ├── auth_service.dart    # login/register/refresh
│   │   └── token_storage.dart   # flutter_secure_storage
│   ├── sse/
│   │   └── sse_client.dart      # yeniden bağlanan SSE tüketicisi
│   ├── trtc/
│   │   └── trtc_service.dart    # enterRoom/exitRoom sarmalayıcı
│   └── cache/
│       └── cache_service.dart   # görsel + veri cache
└── features/                   # Part 2+ ekranlar buraya
```

✓ PART 1 KABUL KRİTERLERİ

1. Dio client + JWT interceptor (7g access / 30g refresh) çalışıyor, 401'de otomatik refresh yapıyor.
2. 6 auth endpoint'i (register/login/refresh/google/apple/tiktok) entegre.
3. `flutter_secure_storage` ile token güvenli saklanıyor.
4. SSE client bir odaya bağlanıp `message / session_ended` olaylarını alıyor.

5. `/api/trtc/token` ile `userSig` alınıp TRTC odasına girilebiliyor.
 6. Standart `{success,data,error}` parse helper'ı tüm isteklerde kullanılıyor.
-

Sonraki: Part 2 — Premium UI/UX (Ana sayfa, Sesli sohbet odaları, Canlı yayın, Keşfet, Gold, Tarot, Burç, kart tasarımları, animasyonlar).